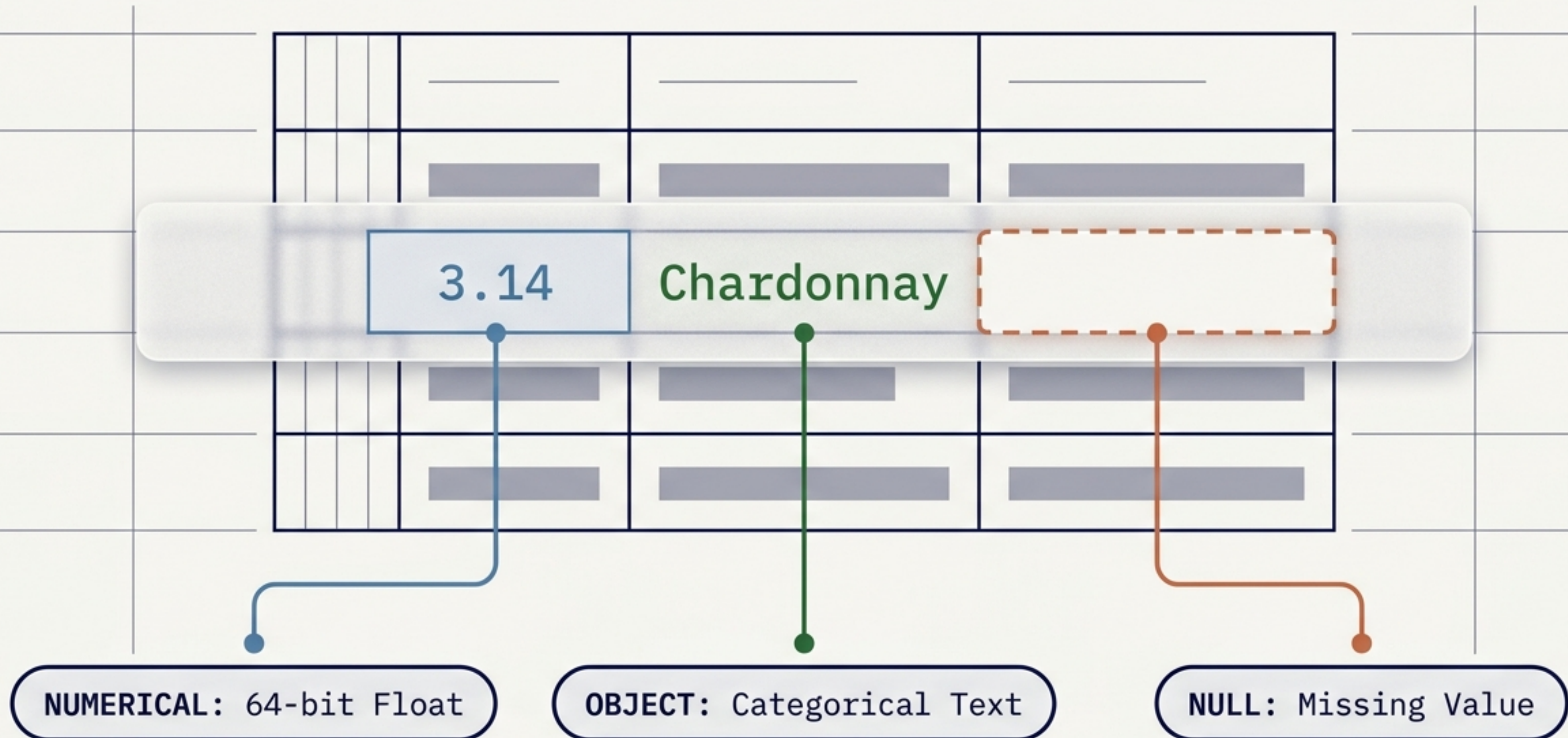


# Mastering Data Types & Missing Values

The essential Pandas workflow for diagnosing, transforming, and refining raw datasets before analysis begins.





# THE TWO PRIMARY PROGRESS BLOCKERS

Raw data is rarely ready for immediate analysis. Before modeling or visualizing, datasets must overcome two structural flaws.

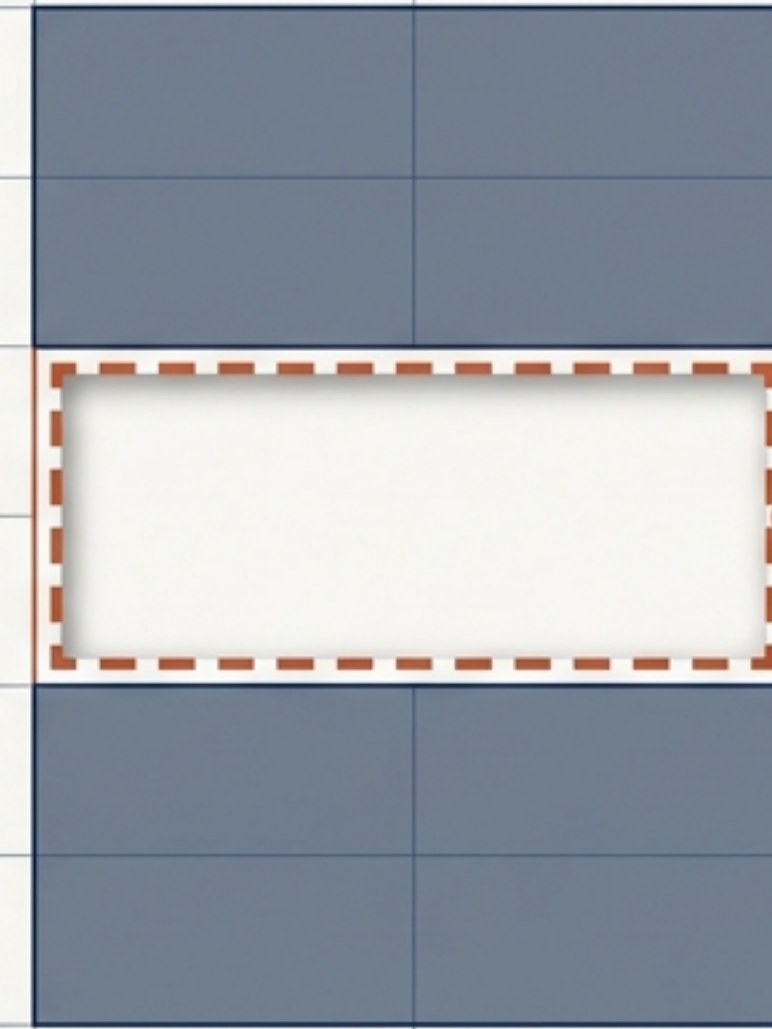
## MISALIGNED TYPES

Data speaking the wrong language.



Data speaking the wrong language.

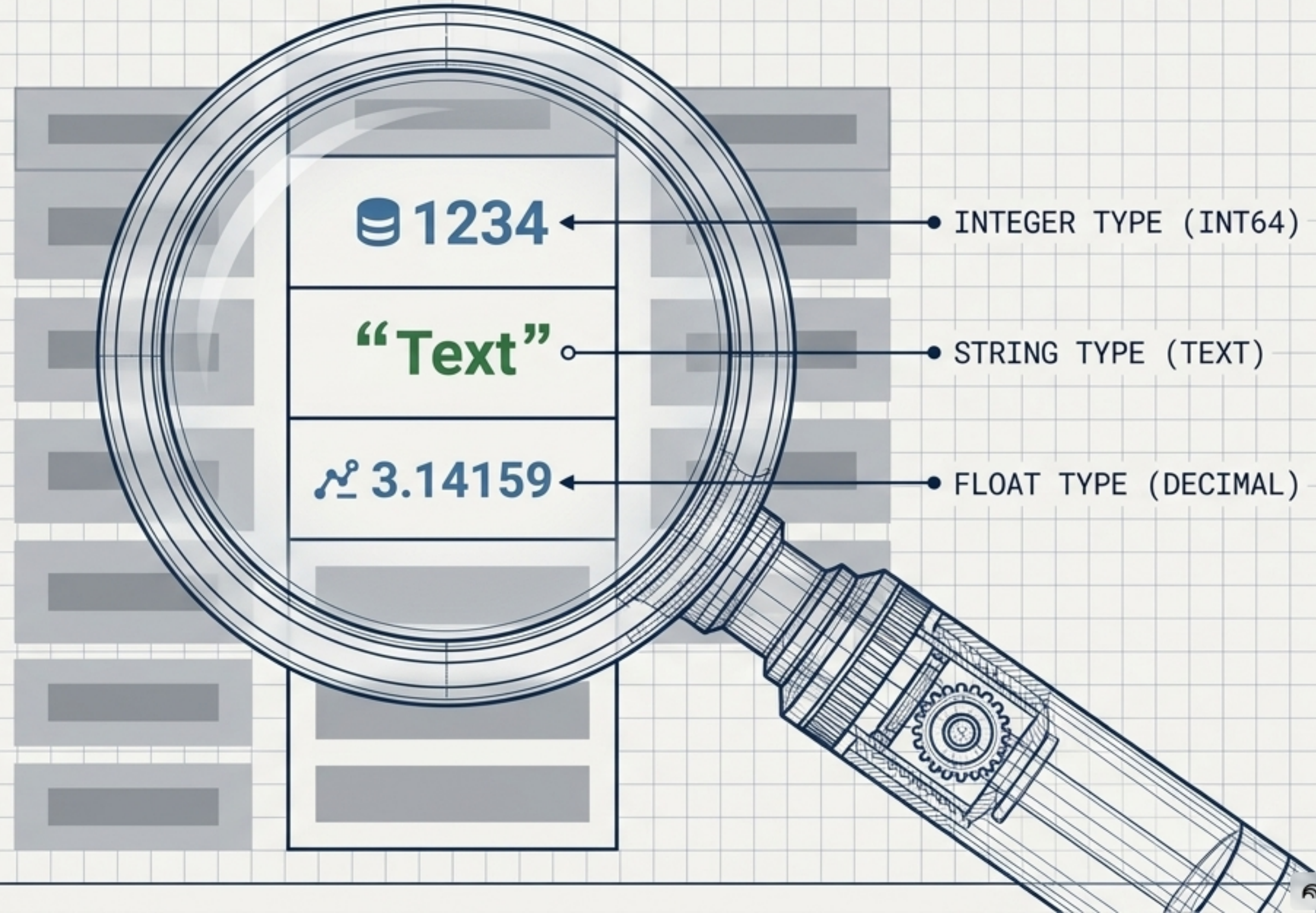
## MISSING DATA



Structural voids in the dataset.



# PART 1: INSPECTING & TRANSFORMING TYPES





# DIAGNOSING DATA ANATOMY WITH DTYPE

The data type tells us how Pandas stores the data internally. You can inspect a single column or scan the entire DataFrame at once.

## Micro View

| price |
|-------|
| 25.00 |
| 40.00 |
| 18.50 |
| 32.00 |
| 55.00 |

`reviews.price.dtype` `dtype('float64')`

## Macro View

| country | description  | points | price |
|---------|--------------|--------|-------|
| US      | Excellent... | 95     | 40.00 |
| France  | Complex...   | 88     | 22.50 |
| Italy   | Vibrant...   | 92     | 35.00 |

`reviews.dtypes`

|             |         |
|-------------|---------|
| country     | object  |
| description | object  |
| points      | int64   |
| price       | float64 |

Note: A DataFrame or Series index has its own dtype, too.



# THE PANDAS CORE TYPES MATRIX

**float64**

3.14

64-bit floating-point  
number.

**int64**

42

Similarly sized integer.

**object**

"Unknown"

Text/String data.

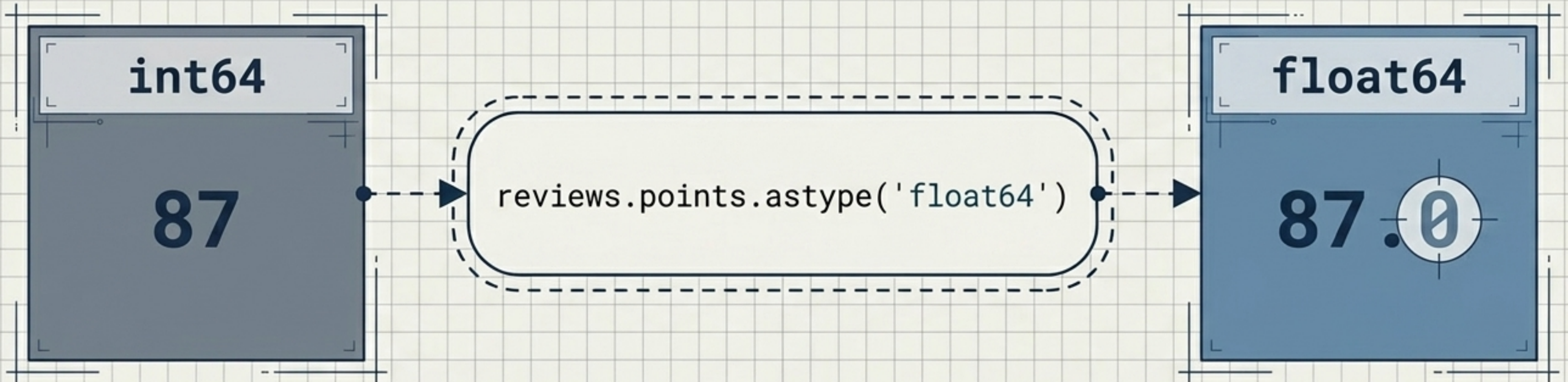
Expert Insight

The String Peculiarity: Columns consisting entirely of strings do not get their own dedicated string type; they are instead assigned the **object** type. •



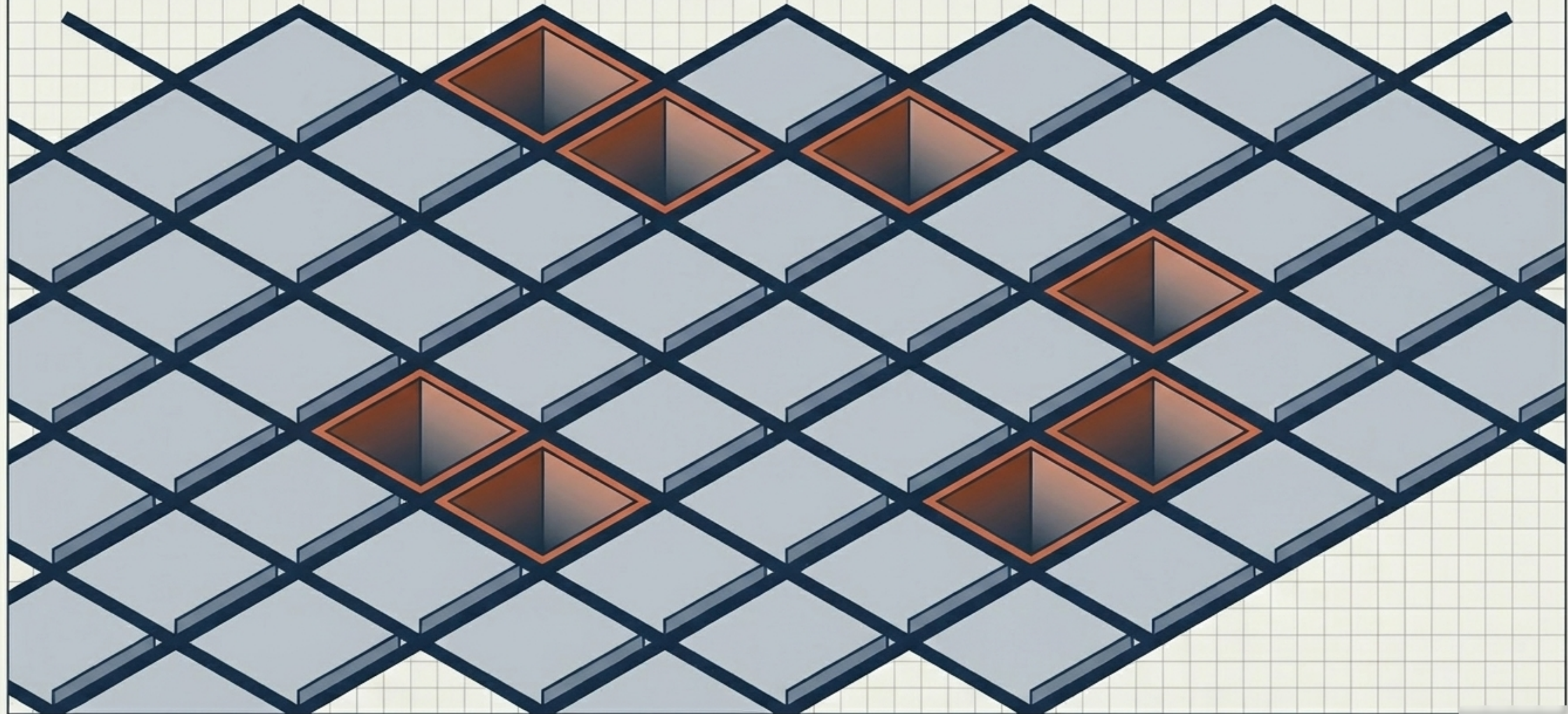
# FORCING CONVERSION WITH ASTYPE

When mathematically or logically appropriate, convert a column from one type to another to unlock analytical functions.





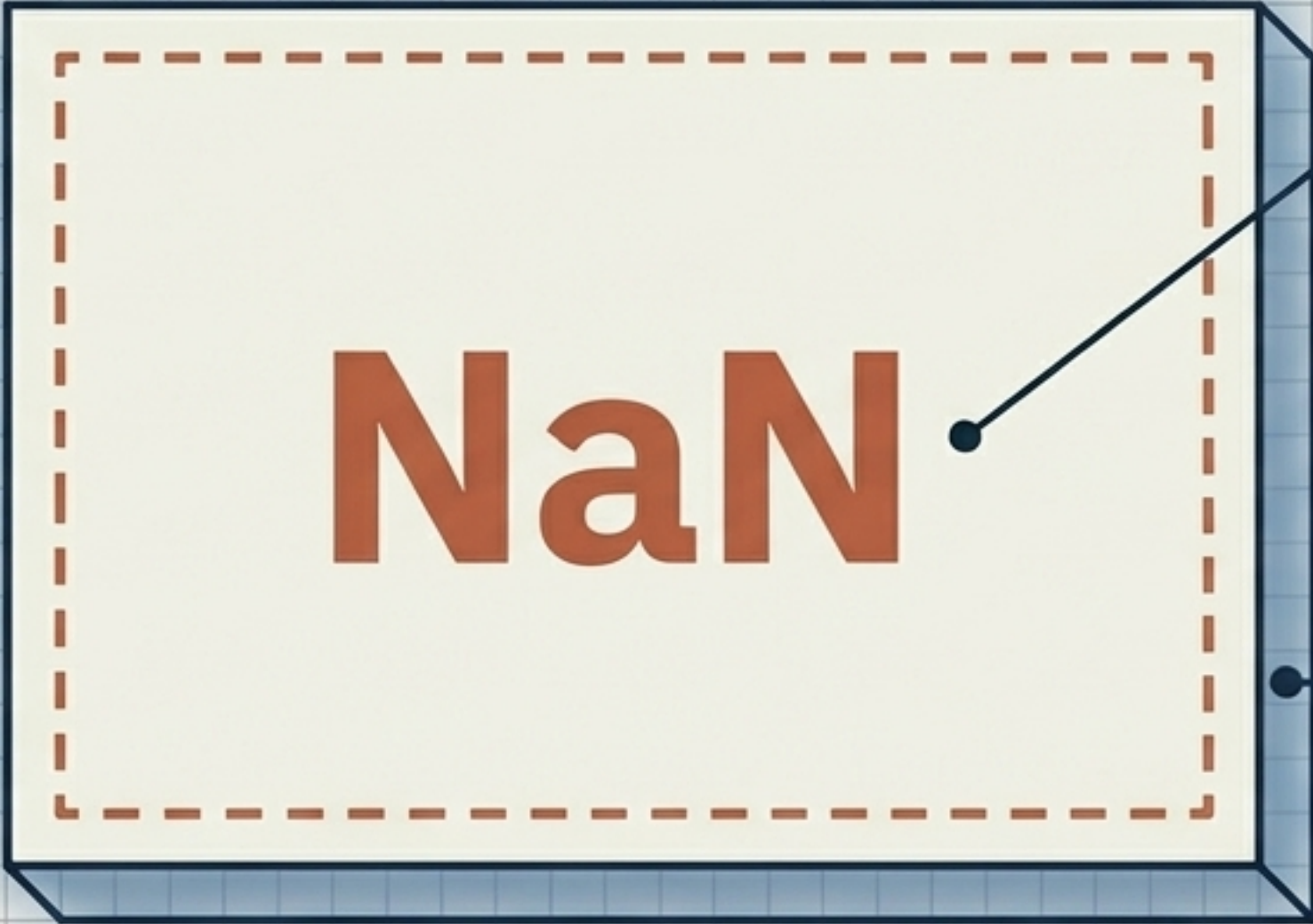
# PART 2: DIAGNOSING & TREATING MISSING DATA





# THE 'NOT A NUMBER' PHENOMENON

Pandas assigns missing entries the value **NaN** (Not a Number).



NaN

Represents a missing or null entry in the dataset.

**Technical Quirk:** For technical reasons, NaN values are **ALWAYS** treated as the **float64** data type.



# ISOLATING THE VOIDS WITH PD.ISNULL

Locate missing data across vast datasets by applying a boolean mask. (Companion function: **pd.notnull**).

**NORMAL VIEW**

| country | points | price |
|---------|--------|-------|
| Italy   | 87     | 30.0  |
| NaN     | 90     | 55.0  |
| France  | 95     | 75.0  |
| NaN     | 92     | 60.0  |

`pd.isnull(reviews.country)`

**MASKED VIEW**

|       |       |       |
|-------|-------|-------|
| False | False | False |
| True  | False | False |
| False | False | False |
| True  | False | False |



# THE MITIGATION MATRIX: FILLNA STRATEGIES

Missing data doesn't dictate deleting the row. Pandas offers strategic mitigation to fill the gaps.

Code: `fillna('Unknown')`

Italy

Unknown

NaN

France



Injects a specific, placeholder value into all NaN voids.

Code: `fillna(method='bfill')`

Italy

NaN

France

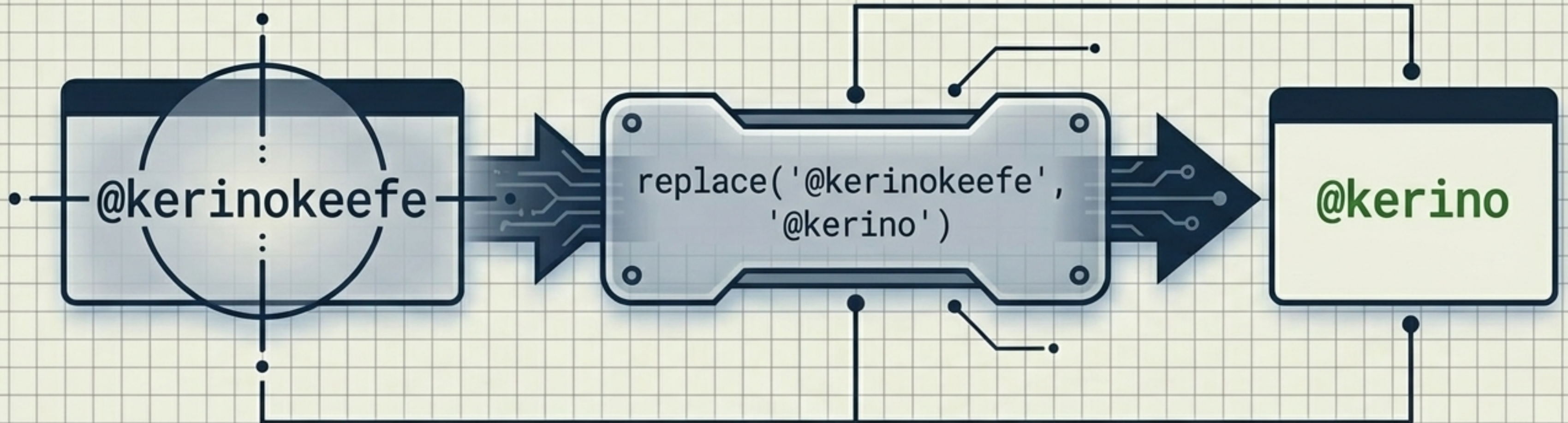


Duplicates the next valid observation backward to fill the void.



# TARGETED REPLACEMENT FOR SENTINEL VALUES

Non-null data can still be incorrect. Use `replace()` to target outdated information or sentinel values like 'Unknown', 'Undisclosed', or 'Invalid'.





# THE DATA INTEGRITY CHEAT SHEET

The complete Pandas data refinery workflow in four sequential steps.

## INSPECT

What language is it?

`dtype`

`dtypes`

## DIAGNOSE

Find the gaps.

`pd.isnull()`

`pd.notnull()`

## TRANSFORM

Force translation.

`astype()`

## MITIGATE

Patch the voids.

`fillna()`

For NaNs

`replace()`

For sentinel values



# INITIALIZE THE REFINERY

The tools are loaded. It is time to apply these commands to live datasets.

